



1. Генерация тестов — это не генерация большого количества проверок

Когда я использую AI для тестирования, моя цель не состоит в том, чтобы получить как можно больше файлов с `test_...`. Количество тестов само по себе ничего не говорит о качестве проверки. Мне важно, какие свойства программы эти тесты подтверждают и какие ошибки они способны обнаружить.

Особенность AI заключается в том, что модель может очень быстро изучить функцию и предложить несколько сценариев. Это удобно, когда необходимо проверить обычный случай, пограничные значения, ошибочные входные данные и разные ветви условий. Но модель видит код таким, каким он написан сейчас. Если реализация содержит ошибочное бизнес-правило, существует риск, что сгенерированный тест просто закрепит эту ошибку как ожидаемое поведение.

Поэтому при работе через Hermes я буду давать агенту не только исходный код. Ему понадобится понимать контракт модуля, требования и существующие правила проекта. После генерации Hermes должен будет запускать тесты в настоящем окружении проекта, а не просто утверждать, что созданный тест выглядит корректно.

2. Unit-тест проверяет небольшую единицу поведения

Unit-тест я использую тогда, когда хочу проверить небольшой участок логики относительно изолированно от остальных частей системы. Такой тест должен быстро запускаться и давать достаточно точный ответ о том, где возникла проблема.

Например, если у нас есть функция расчёта скидки, для unit-теста мне не нужна настоящая PostgreSQL, HTTP-сервер или платёжная система. Мне важно передать входные значения и проверить результат вычисления.

При генерации таких тестов AI особенно полезен для анализа условий внутри функции. Если код содержит несколько веток, агент может увидеть, что одна ветка выполняется для обычного пользователя, другая для VIP-клиента, а третья для ошибочного значения.

Но хороший unit-тест проверяет поведение, а не внутреннее устройство реализации. Если тест напрямую зависит от каждой локальной переменной или от точной последовательности внутренних вызовов без необходимости, обычный рефакторинг начнёт ломать тесты даже тогда, когда внешнее поведение осталось правильным.

3. Integration-тест проверяет взаимодействие компонентов

Integration-тест нужен уже в другой ситуации. Здесь меня интересует не одна функция, а корректность взаимодействия нескольких частей системы.

Например, repository может быть написан правильно сам по себе, SQL-запрос тоже выглядит правильно, но ошибка возникает только при взаимодействии с настоящей схемой базы данных. Именно такую проблему изолированный unit-тест может не обнаружить.

Поэтому integration-тест обычно работает с более реалистичной средой. В зависимости от проекта это может быть тестовая база данных, HTTP-приложение, файловая система или несколько внутренних сервисов.

AI при генерации integration-тестов должен понимать эту границу. Если агент подменил mock-объектами вообще все зависимости, фактически он снова создал unit-тест. Если же для простой функции он начинает поднимать половину инфраструктуры приложения, тест становится неоправданно тяжёлым.

В Hermes агент сможет запускать реальные команды проекта через terminal-tools, поэтому на практике мы сможем отличать тест, который только был написан моделью, от теста, который действительно запускается в существующей среде. Hermes предоставляет агентам файловые и терминальные инструменты именно для работы с реальным проектом.

4. Покрытие показывает, какой код был выполнен тестами

Когда мы начинаем автоматически генерировать тесты, возникает соблазн использовать процент покрытия как главный показатель результата. Например, получить 90 процентов и решить, что модуль протестирован хорошо.

Покрытие означает другое. Оно показывает, какие элементы программы выполнялись во время тестирования. Если строка ни разу не выполнялась, это полезный сигнал: определённая часть реализации осталась без проверки.

Но выполнение строки ещё не означает, что тест действительно проверил её правильность. Я могу вызвать функцию, пройти через весь метод и вообще не проверить итоговый результат. Формально coverage увеличится, но полезного контроля поведения почти не появится.

Поэтому при AI-генерации я рассматриваю coverage как навигационный инструмент. Он помогает найти непроверенные области, но не заменяет анализ качества самих утверждений теста.

5. Покрытие ветвей обычно информативнее одного line coverage

Особенно важный случай возникает с условными конструкциями.

Представим, что строка с `if` была выполнена во время теста. Line coverage может считать её покрытой. Но одна ветвь условия могла ни разу не выполняться.

Поэтому для кода с большим количеством условий мне интересен branch coverage. Он показывает, проходили ли тесты по различным направлениям выполнения программы.

Для AI это полезная обратная связь. Агент может сначала сгенерировать набор тестов, затем получить отчёт покрытия и увидеть, что одна ветка осталась недостижимой текущим набором сценариев.

Но я не прошу модель механически закрывать каждую ветку любой ценой. Иногда ветка вообще является недостижимой из-за устаревшего кода. Иногда для её проверки приходится создавать искусственный тест, который ничего не говорит о реальном поведении системы. Тогда уже возникает вопрос о качестве самой реализации.

6. Фикстура создаёт известное начальное состояние

Чтобы тест был воспроизводимым, мне необходимо заранее определить его исходное состояние. Для этого используются фикстуры.

Фикстура может создавать тестового пользователя, заказ, набор товаров, временный файл или подготовленное состояние базы данных. Главное требование состоит в том, чтобы тест получил известные данные и не зависел от случайного состояния предыдущего запуска.

AI удобно использовать для генерации фикстур, особенно когда модели данных достаточно объёмные. Агент может посмотреть структуру сущности и подготовить минимальный корректный объект для тестов.

Но здесь я контролирую объём данных. Если для проверки функции требуется только цена и количество, нет смысла генерировать огромный объект со всеми полями production-системы. Чем тяжелее фикстура, тем сложнее понять, какие данные действительно влияют на тест.

В следующей практике Hermes сможет не только создать такие данные, но и проверить их вместе с реальными тестами. Это важнее, чем получить синтаксически красивую фикстуру без запуска.

7. Mock нужен там, где зависимость мешает проверить нашу логику

При unit-тестировании часто приходится работать с внешними зависимостями. Например, функция вызывает платёжный сервис, отправляет email или обращается к внешнему API.

Мне не нужно совершать настоящий платёж каждый раз, когда запускаются unit-тесты. Поэтому такую зависимость можно заменить mock-объектом.

Но mock я использую не потому, что «в тестах всё нужно подменять». Я подменяю именно границу, которая не относится к проверяемой логике или делает тест медленным, нестабильным либо опасным.

AI иногда склонен слишком активно использовать mocking. В итоге тест проверяет набор искусственных объектов и почти не соприкасается с реальной программой. Поэтому при генерации я буду отдельно задавать, какие зависимости можно подменять, а какие должны оставаться настоящими.

Для integration-тестов эта граница обычно сдвигается. Например, repository уже имеет смысл проверить на настоящей тестовой базе, а внешний платёжный провайдер всё ещё можно заменить контролируемым тестовым аналогом.

8. Мутационное тестирование проверяет силу наших тестов

Теперь рассмотрим более интересную проверку качества тестов — mutation testing.

Идея здесь обратная обычному тестированию. Мы намеренно слегка повреждаем программу и смотрим, обнаружат ли существующие тесты изменение.

Например, условие `age >= 18` можно временно изменить на `age > 18`. Если все тесты после этого продолжают проходить, значит они не различают два варианта поведения и, вероятно, недостаточно точно проверяют границу.

Такое искусственное изменение называют мутантом. Если тесты падают, мутант считается обнаруженным. Если тесты продолжают проходить, у нас появляется сигнал, что конкретное изменение поведения осталось незамеченным.

Для AI это очень полезная обратная связь. Вместо запроса «добавь ещё тестов» я могу показать агенту конкретный выживший мутант. Тогда задача становится намного точнее: необходимо понять, какого сценария не хватает, чтобы неправильное изменение было обнаружено.

При этом mutation testing не заменяет обычные unit- и integration-тесты. Это уже способ оценить чувствительность существующего набора проверок.

9. Сгенерированный тест тоже требует review

Я не отношусь к тесту, созданному моделью, как к автоматически правильному коду.

AI может написать тест, который всегда проходит. Он может вычислить ожидаемый результат тем же алгоритмом, который используется внутри тестируемой функции. Он может проверить не то требование или создать фикстуру, которая никогда не встречается в реальной системе.

Поэтому при review теста я смотрю, какое поведение подтверждается и способен ли тест реально упасть после соответствующей ошибки в production-коде.

Особенно подозрительно выглядит ситуация, когда AI одновременно изменяет реализацию и тест, а затем сообщает, что всё прошло успешно. В таком случае модель потенциально способна изменить обе стороны так, чтобы они согласовались между собой, но уже не соответствовали исходному требованию.

Именно поэтому в Hermes удобно разделять анализ, генерацию и запуск. Агент может прочитать модуль, создать тестовый файл, запустить test runner и получить фактический результат. При необходимости для такого процесса можно использовать существующие software-development Skills; например, Hermes сейчас включает bundled Skill `test-driven-development`, а Skills вообще предназначены для закрепления повторяемых процедур работы агента.

10. Hermes должен замкнуть цикл «сгенерировал — запустил — проанализировал»

Для нашей дальнейшей работы главное отличие агентного подхода от обычного генератора кода заключается в наличии обратной связи.

Я не хочу получать от Hermes только текст файла `test_service.py`. После создания тестов агент должен запустить их тем же test runner, которым пользуется проект, получить реальные сообщения об ошибках и определить причину результата.

Если тест не проходит, агенту необходимо различить несколько ситуаций. Ошибка может находиться в production-коде, в самом тесте, в неправильно созданной фикстуре или в окружении. Автоматически переписывать тест только ради зелёного результата нельзя.

Если тесты прошли, это тоже ещё не завершает анализ. Дальше можно посмотреть coverage и определить, какие важные ветви остались без проверки. Для более строгой оценки можно использовать mutation testing и посмотреть, способны ли созданные тесты действительно обнаруживать изменения поведения.

Hermes подходит для такого процесса потому, что его агент может сочетать чтение и изменение файлов с запуском инструментов проекта через terminal, а процедуры тестирования можно закреплять в Skills и повторно применять к следующим модулям.

В следующей практической работе мы реализуем именно такой цикл: возьмём готовый Python-модуль, дадим Hermes задачу сначала разобраться в его контракте, затем сгенерировать unit- и integration-тесты, подготовить необходимые фикстуры, реально запустить тесты и после этого проанализировать результаты и покрытие. Отдельно проверим, способен ли наш набор тестов обнаружить намеренно изменённое поведение программы.